

Sprint Documentation

Sprint 25	Start Date: 25/08/2026	Final Date: 07/10/2026
Project Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
João Pedro Chapichi		
João Antonio Gomes Cestonaro		

Sprint Backlog

Task #	Description	Start Date	Final date
#82	Validações de UnidadeMedida	25/08/2026	07/10/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#82	Garantir que o cadastro e manutenção das unidades de medida possuam validações completas de integridade, padronização e regras de negócio, assegurando consistência no uso em tipos de despesa e lançamentos no sistema. Além disso, aplicar correções necessárias no fluxo atual.	João Pedro Chapichi, João Antonio Gomes Cestonaro	Completo	2 horas	2 horas

Sprint Description

Tarefa: Validações de Unidade de Medida

Objetivo

Garantir que o cadastro e manutenção das unidades de medida possuam validações completas de integridade, padronização e regras de negócio, assegurando consistência no uso em tipos de despesa e lançamentos no sistema. Além disso, aplicar correções necessárias no fluxo atual.

Descrição

Implementar validações no backend para os campos da entidade de unidade de medida, assegurando:

- Dados obrigatórios
- Padronização de informações
- Integridade relacional
- Retorno padronizado de erros (lista completa)

CrITÉrios de Aceite

Campos Obrigatórios

-
- Validar que os campos obrigatórios não estão vazios:
 - descricao
 - abreviatura
 - situacao

Validação de Descrição

- Campo obrigatório

Mínimo de 3 caracteres

Máximo de 150 caracteres

Remover espaços extras (trim + normalização de espaços)

Padronizar capitalização para evitar variações inconsistentes

Não permitir descrições duplicadas no sistema (case-insensitive)

Validação de Abreviatura

- Campo obrigatório

Mínimo de 1 caractere

Máximo de 45 caracteres

Remover espaços extras (trim)

Padronizar formato (ex.: caixa consistente)

Não permitir abreviaturas duplicadas no sistema (case-insensitive)

Validação de Situação

- Deve corresponder a um valor válido do enum Situacao
- Valores permitidos:

- 1 = Ativo
- 2 = Inativo

Rejeitar valores fora do padrão

Regra de Negócio (Situação)

-
- Unidades de medida INATIVAS:
 - Não devem aparecer para seleção em cadastro de tipo de despesa
 - Não podem ser utilizadas em novos lançamentos

Validação de Inativação

-
- Não permitir inativação de unidade de medida que possua:
 - Tipos de despesa vinculados ativos

Validação em Atualização

- Ao alterar situacao para INATIVO via PUT, aplicar a mesma regra de bloqueio da inativação

Garantir consistência entre os endpoints PUT e PATCH

Manter consistência dos dados após alteração

Normalização e Boas Práticas

- Implementar validações na camada **SERVICE** (UnidadeMedidaService)

Não depender apenas de **DataAnnotations**

Retornar mensagens claras ao usuário

Retornar os erros em **lista única** com todos os problemas de uma só vez

Criar exceção específica (ex.: UnidadeMedidaValidationException) e tratar no controller com 400

Evitar retorno de stack trace para erro de validação

Ajustes Técnicos Esperados

- Sobrescrever Create e Update em UnidadeMedidaService, seguindo padrão de validação centralizada
- Evoluir IUnidadeMedidaRepository para suportar:
 - checagem de duplicidade por descrição normalizada
 - checagem de duplicidade por abreviatura normalizada

- checagem de vínculos ativos com TipoDespesa

Adequar UnidadeMedidaController para tratar exceções de validação e retornar ResponseEnum.INVALID

Incluir testes automatizados das regras críticas (obrigatórios, enum, duplicidade, inativação com vínculo)

Observações e Sugestões

- Evitar duplicidade de unidades (ex.: "Quilowatt-hora" vs "quilowatt-hora")
- Evitar duplicidade de abreviação (ex.: "kWh" vs "KWH")
- Centralizar validações para facilitar manutenção futura
- Garantir consistência entre regras de negócio e API
- Revisar compatibilidade entre limites do service e constraints do banco (Builder)
- Verificar o projeto antes do envio

Sprint Results

O que foi feito:

1. Unidade de medida agora valida, normaliza e agrega todos os erros antes de persistir.
 - Descrição: Obrigatória, 3-150 caracteres, espaços normalizados e capitalização padronizada.
 - Abreviatura: Obrigatória, até 45 caracteres, espaços removidos e padronizada em caixa alta.
 - Situação: aceita somente 1 (Ativo) ou 2 (Inativo).
 - Duplicidades de descrição e abreviaturas bloqueadas sem diferenciar maiúsculas/minúsculas.
2. Nova UnidadeMedidaValidationException, devolvida como 400 / ResponseEnum.INVALID, com lista única de erros e sem Stacktrace para validação.
3. PUT e PATCH aplicam a mesma regra: Não permite inativar unidade vinculada a tipos de despesa ativos.
4. Repositório evoluído com checagens de duplicidade normalizada e vínculos ativos.
5. Limite do banco/EF para descrição alinhado a 150 caracteres.
6. O fluxo já existente de TipoDespesaService continua impedindo o uso de unidade inativa em novos tipos de despesa;
7. Adicionados 4 testes de endpoint cobrindo erros agregados, enum, normalização, duplicidade, e inativação por PUT/PATCH.
8. Adicionando teste para a classe UnidadeMedida.

UnidadeMedidaService:

```
using AutoMapper;
using Civitas.WebAPI.Data.Interfaces;
using Civitas.WebAPI.Objects.Dtos.Entities;
using Civitas.WebAPI.Objects.Enums;
using Civitas.WebAPI.Objects.Models;
using Civitas.WebAPI.Services.Interfaces;
using Civitas.WebAPI.Services.Validation;
using System.Globalization;
using System.Text.RegularExpressions;

namespace Civitas.WebAPI.Services.Entities
{
    /// <summary>Centraliza o cadastro, a normalização e as regras de negócio das unidades de medida.</summary>
    public class UnidadeMedidaService : GenericService<UnidadeMedida, UnidadeMedidaDTO>, IUnidadeMedidaService
    {
        private static readonly CultureInfo PtBrCulture = new("pt-BR");
        private readonly IUnidadeMedidaRepository _unidadeMedida;
        private readonly IMapper _mapper;

        public UnidadeMedidaService(IUnidadeMedidaRepository unidadeMedida, IMapper mapper)
            : base(unidadeMedida, mapper)
        {
            _unidadeMedida = unidadeMedida;
            _mapper = mapper;
        }

        public override async Task Create(UnidadeMedidaDTO entityDTO)
        {
            ValidateDtoInstance(entityDTO);
            Normalize(entityDTO);

            var errors = ValidateCommonRules(entityDTO);
            await ValidateBusinessRules(entityDTO, errors);
            ThrowIfInvalid(errors);

            var entity = _mapper.Map<UnidadeMedida>(entityDTO);
            await _unidadeMedida.Add(entity);
            entityDTO.Id = entity.Id;
        }
    }
}
```

```
12  {
15  {
40
41  public override async Task Update(UnidadeMedidaDTO entityDTO, int id)
42  {
43      ValidateDtoInstance(entityDTO);
44
45      var existingUnidadeMedida = await _unidadeMedida.GetById(id);
46      if (existingUnidadeMedida is null)
47      {
48          throw new KeyNotFoundException($"Unidade de medida com id {id} não encontrada.");
49      }
50
51      Normalize(entityDTO);
52
53      var errors = ValidateCommonRules(entityDTO);
54      await ValidateBusinessRules(entityDTO, errors, id, existingUnidadeMedida);
55      ThrowIfInvalid(errors);
56
57      var entity = _mapper.Map<UnidadeMedida>(entityDTO);
58      entity.Id = id;
59      entity.Excluido = existingUnidadeMedida.Excluido;
60      entity.DataExclusao = existingUnidadeMedida.DataExclusao;
61      await _unidadeMedida.Update(entity);
62      entityDTO.Id = id;
63  }
64
65  private static void ValidateDtoInstance(UnidadeMedidaDTO? dto)
66  {
67      if (dto is null)
68      {
69          throw new UnidadeMedidaValidationException(["O corpo da requisição é obrigatório."]);
70      }
71  }
72
73  private static void Normalize(UnidadeMedidaDTO dto)
74  {
75      var descricao = NormalizeSpaces(dto.Descricao);
76      dto.Descricao = string.IsNullOrEmpty(descricao)
```

```

12  {
15  {
16  },
72
73  private static void Normalize(UnidadeMedidaDTO dto)
74  {
75      var descricao = NormalizeSpaces(dto.Descricao);
76      dto.Descricao = string.IsNullOrEmpty(descricao)
77          ? descricao
78          : PtBrCulture.TextInfo.ToTitleCase(descricao.ToLower(PtBrCulture));
79      dto.Abreviatura = NormalizeSpaces(dto.Abreviatura).ToUpperInvariant();
80  }
81
82  private static List<string> ValidateCommonRules(UnidadeMedidaDTO dto)
83  {
84      var errors = new List<string>();
85
86      if (string.IsNullOrWhiteSpace(dto.Descricao))
87      {
88          errors.Add("O campo Descrição é obrigatório.");
89      }
90      else
91      {
92          if (dto.Descricao.Length < 3) errors.Add("O campo Descrição deve ter no mínimo 3 caracteres.");
93          if (dto.Descricao.Length > 150) errors.Add("O campo Descrição deve ter no máximo 150 caracteres.");
94      }
95
96      if (string.IsNullOrWhiteSpace(dto.Abreviatura))
97      {
98          errors.Add("O campo Abreviatura é obrigatório.");
99      }
100     else if (dto.Abreviatura.Length > 45)
101     {
102         errors.Add("O campo Abreviatura deve ter no máximo 45 caracteres.");
103     }
104
105     if (!Enum.IsDefined(dto.Situacao) || dto.Situacao is not (Situacao.ATIVO or Situacao.INATIVO))
106     {
107         errors.Add("Situação inválida. Valores permitidos: 1 (Ativo) ou 2 (Inativo).");
108     }
109 }
110
111 private async Task ValidateBusinessRules(
112     UnidadeMedidaDTO dto,
113     ICollection<string> errors,
114     int? id = null,
115     UnidadeMedida? existing = null)
116 {
117     if (!string.IsNullOrWhiteSpace(dto.Descricao) &&
118         await _unidadeMedida.ExistsByDescricaoNormalized(NormalizeForComparison(dto.Descricao), id))
119     {
120         errors.Add("Já existe uma unidade de medida cadastrada com esta descrição.");
121     }
122
123     if (!string.IsNullOrWhiteSpace(dto.Abreviatura) &&
124         await _unidadeMedida.ExistsByAbreviaturaNormalized(NormalizeForComparison(dto.Abreviatura), id))
125     {
126         errors.Add("Já existe uma unidade de medida cadastrada com esta abreviatura.");
127     }
128
129     if (id.HasValue && existing?.Situacao == Situacao.ATIVO && dto.Situacao == Situacao.INATIVO &&
130         await _unidadeMedida.HasTiposDespesaAtivosVinculados(id.Value))
131     {
132         errors.Add("Não é permitido inativar unidade de medida com tipos de despesa ativos vinculados.");
133     }
134 }
135
136 private static void ThrowIfInvalid(ICollection<string> errors)
137 {
138     if (errors.Count > 0) throw new UnidadeMedidaValidationException(errors);
139 }
140
141 private static string NormalizeSpaces(string? value) => Regex.Replace(value?.Trim() ?? string.Empty, "\\s+", " ");
142 private static string NormalizeForComparison(string value) => NormalizeSpaces(value).ToUpperInvariant();
143 }

```

UnidadeMedidaRepository:

```

namespace Civitas.WebAPI.Data.Repositories
{
    public class UnidadeMedidaRepository : GenericRepository<UnidadeMedida>, IUnidadeMedidaRepository
    {
        private readonly AppDbContext _context;
        public UnidadeMedidaRepository(AppDbContext context) : base(context)
        {
            _context = context;
        }

        public async Task<bool> ExistsByDescricaoNormalized(string descricaoNormalizada, int? ignoreId = null)
        {
            var query = _context.UnidadesMedida.AsNoTracking().Where(u => !u.Excluido);
            if (ignoreId.HasValue) query = query.Where(u => u.Id != ignoreId.Value);

            return (await query.Select(u => u.Descricao).ToListAsync())
                .Any(descricao => NormalizeForComparison(descricao) == descricaoNormalizada);
        }

        public async Task<bool> ExistsByAbreviaturaNormalized(string abreviaturaNormalizada, int? ignoreId = null)
        {
            var query = _context.UnidadesMedida.AsNoTracking().Where(u => !u.Excluido);
            if (ignoreId.HasValue) query = query.Where(u => u.Id != ignoreId.Value);

            return (await query.Select(u => u.Abreviatura).ToListAsync())
                .Any(abreviatura => NormalizeForComparison(abreviatura) == abreviaturaNormalizada);
        }

        public Task<bool> HasTiposDespesaAtivosVinculados(int idUnidadeMedida)
        {
            return _context.TiposDespesa.AsNoTracking().AnyAsync(t =>
                t.IdUnidadeMedida == idUnidadeMedida &&
                t.Situacao == Situacao.ATIVO &&
                !t.Excluido);
        }

        private static string NormalizeForComparison(string value)
        {
            return Regex.Replace(value.Trim(), "\\s+", " ").ToUpperInvariant();
        }
    }
}

```

Assinaturas

Documento assinado digitalmente
gov.br JOAO PEDRO CHAPICHI
 Data: 26/09/2026 23:29:33-0300
 Verifique em <https://validar.iti.gov.br>

Documento assinado digitalmente
gov.br JOAO ANTONIO GOMES CESTONARO
 Data: 26/09/2026 23:34:20-0300
 Verifique em <https://validar.iti.gov.br>